THE AI TOOLBOX · CLASS 05

Data-Analysis Tools (Python, Gently)

Handle real data with a few lines of Python — **no fear required**.

Merit Point Academy · 60 min · press **S** notes · → next · 💬 to comment

RECAP · WHERE WE LEFT OFF

Last class in 60 seconds

Prompt anatomy

Role · context · format · constraints — and constraints make answers checkable.

The framework

Every paper in three parts: **summary · contribution · limitation** — the limitation is your idea mine.

The seatbelt

LLMs predict plausible text → **verify every claim**. Unverifiable = unusable.

Homework share: who caught a hallucination in the wild? And show a ✓-verified summary. Today your Class-4 assistant becomes a **coding co-pilot** — paste any error, ask what it means, fix it yourself.

AGENDA · HOW THIS HOUR RUNS

Where we're going

- **0–8'** Homework share + the analyst who built his own tool
- **8–18'** Everything is a table · Series & DataFrame
- **18–30'** The four verbs: load · inspect · select · summarize
- **30–38'** Filtering with conditions · handling missing values
- **38–46'** Demos: Colab workflow · the real Titanic · AI-debugging
- **46–54'** Two labs: the AV detection log · Titanic survival, for real
- **54–60'** Your first data task, homework & wrap-up
- Quizzes & 💬 comments throughout

By the end: read any dataset as rows × columns, run pandas' four verbs on real data, and produce your first cleaned dataset + labeled chart in Colab.

WARM-UP · ACTIVATE

Quick check · tap an answer

A self-driving car 'sees' 30 frames every second, detecting several objects per frame. What does one second of its vision look like, written down?

- A movie file — you can't write vision down
- A table: one row per detected object, columns like frame, class, confidence, distance
- A single number between 0 and 1
- Six pages of if-statements

THE STORY

Built to tame a mountain of data

> In 2008, an analyst named **Wes McKinney** was drowning in financial spreadsheets at a hedge fund. Python had no good way to handle tables of data — so he **built one himself** and called it **pandas**. He open-sourced it, and today it's the tool millions of scientists reach for first. It exists because one person was tired of wrestling messy tables by hand.

🔊 **Speak this story** Neural voice · click to play / pause

The name isn't the animal: **pan**(el) **da**(ta) — economists' word for tables over time. Think of it as a super-Excel that speaks Python: millions of rows, automated, reproducible.

CONCEPT 1 · THE ONE SHAPE

Everything is a table

- **Rows = examples.** One detection, one house, one passenger, one day of your life.
- **Columns = variables.** Price, confidence, age, hours slept — one fact per column.
- In pandas this shape is called a **DataFrame**.

Recognize it? It's Class 2's "picture the data table" — today the picture becomes an object you can compute on.

frame	object	conf	dist_m
001	pedestrian	0.94	8.2
001	car	0.88	15.0
002	sign	0.79	22.4

→ a row = one example · ↓ a column = one variable

KEY IDEA — rows are **things**; columns are **facts about them**. See data this way and the code becomes simple. That's 90% of data science.

CONCEPT 2 · THE TWO CONTAINERS

Series and DataFrame

📏 Series — one labeled column

```
scores = pd.Series([98, 87, 92],
  index=["Alice", "Bob", "Carl"])
```

A single column with row labels. Ask it things: `scores.mean()` → 92.3.

🗂️ DataFrame — the whole table

```
df = pd.DataFrame({"name": [...],
  "score": [...], "grade": [...]})
```

Many Series sharing the same rows. Select one column — `df["score"]` — and you get a Series back.

The mental model: DataFrame = the Excel sheet · Series = one of its columns. Two containers, and you already understand both — that's the whole type system you need this course.

CHECK · TABLE THINKING

Quick check · tap an answer

You track your study sessions for a month: each session's date, subject, minutes, and focus rating. In the table, what is one ROW?

- One subject, like math
- One study session — with its date, subject, minutes, and rating as its facts
- One month
- The minutes column

CONCEPT 3 · 80% OF DATA WORK

pandas in four verbs

1 · Load

```
df = pd.read_csv("data.csv")
```

One line: file → DataFrame. (Excel, JSON, SQL — same idea, different verb ending.)

2 · Inspect

```
df.head()  df.info()  df.describe()
```

First five rows · column types & missing counts · statistics of every numeric column.

3 · Select

```
df["conf"]  df[["frame","conf"]]
```

Grab one column (a Series) or several (a smaller DataFrame).

4 · Summarize

```
df["conf"].mean()  df["object"].value_counts()
```

Averages, counts, extremes — the numbers your research question actually needs.

Load → inspect → select → summarize. Four verbs ≈ 80% of all data work. Everything else is variations.

CONCEPT 4 · ASKING QUESTIONS OF THE TABLE

Filtering: keep only the rows that match

One condition

```
sure = df[df["conf"] > 0.9]
```

Read it inside-out: `df["conf"] > 0.9` marks each row True/False; the outer `df[...]` keeps the True ones.

Combined conditions

```
df[(df["object"]=="pedestrian")
  & (df["conf"] < 0.7)]
```

& = and · **|** = or · **~** = not — each condition in its own parentheses.

Why it matters: every research question is a filter. "How does the detector do on pedestrians at night?" = *filter rows, then summarize them*. The right-hand example finds exactly the low-confidence pedestrians your Class-2 strong question worries about.

CONCEPT 5 · REAL DATA HAS HOLES

Missing values: find, then drop or fill

🔎 Find

```
df.isnull().sum()
```

Missing-value count per column — always the **first diagnostic** after loading.

🗑️ Drop

```
df = df.dropna()
```

Delete rows with holes. Fine when few rows are affected — simple and honest.

🩹 Fill

```
df["age"] = df["age"]
  .fillna(df["age"].median())
```

Replace holes with the median/mean. Keeps the rows when too many would die.

Drop or fill? A judgment call — few holes: drop; many holes: fill, and **say so in your write-up**. Silently invented data is a small cousin of Class 4's hallucination: always disclose what you filled.

CHECK · DROP OR FILL

Quick check · tap an answer

Your 900-row dataset has 12 rows missing 'distance' and 400 rows missing 'age'. What's the sensible plan?

- Drop every row that has any hole — 488 rows left is fine
- Fill everything with 0 so the code runs
- Drop the 12 distance holes; fill the 400 ages with the median — and disclose the fill
- Delete both columns entirely

ZOOM OUT · WHY PYTHON FOR ALL OF THIS

The language your whole toolbox speaks

📖 Reads like English

`df["age"].mean()` — you can guess what that does. Energy goes to the research, not the syntax.

🧰 Batteries included

pandas (tables) · NumPy (math) · Matplotlib (charts) · scikit-learn (models — you met it in Class 3's lab). One ecosystem, every pipeline stage.

🌍 Everyone's here

The world's largest data community: every error you'll ever hit has been answered — and your Class-4 assistant read all of it.

🚀 Grows with you

The same language runs TensorFlow and PyTorch — today's `read_csv`, someday your neural network. Nothing to relearn.

CASE STUDY · OUR RUNNING EXAMPLE, AS DATA

Reading the car's diary

A perception system's output **is** a CSV — one row per detected object. Walk it with the four verbs:

1. **Columns**

   frame · object_class · confidence · box size · distance_m

2. **Inspect**

   df.head() shows the shape; df.describe() → average confidence?

3. **Filter**

   Only pedestrians · only conf < 0.7 — the risky detections

4. **Summarize**

   value_counts() → what does the car see most? Objects per frame?

The thread continues: Class 2 asked whether night data raises pedestrian recall; Class 3 drew its pipeline; today you're inside the pipeline's **data stage**, touching the actual rows. Prefer another domain? Any log, any diary, any sensor — same four verbs.

DISCUSS · 5 MINUTES · DESIGN A TABLE

Your week as a dataset

Design the table for a dataset about **your own life this week** — sleep, mood, screen time, study, sport… anything measurable.

Decide

What is one **row** — a day? a study session? a meal? Which 4–6 **columns** would you track?

Stress-test

Which column would be hardest to measure **honestly**? (Mood on a 1–10 scale? Screen time you'd rather not know?)

Pairs, 3 minutes — then a few tables out loud. Post your columns in 💬.

LIVE DEMO 1

Colab — the full first-data-task flow

Tonight's deliverable, performed live: file to chart in five moves. Watch once, then it's yours.

```
You ▸ Upload titanic.csv (folder icon → upload) — or read it straight from a URL
You ▸ df = pd.read_csv("titanic.csv") · df.head() · df.info()
You ▸ df.isnull().sum() → decide: dropna() or fillna(median)
You ▸ df.describe() — screenshot-worthy stats appear
You ▸ df["Age"].plot(kind="hist", title="Passenger ages"); label the axes!
```

colab.research.google.com

LIVE DEMO 2

Kaggle — meet the real Titanic

Class 1's Lab 1 used 12 toy rows. The real dataset: 891 passengers, 12 columns, genuine holes in the Age column.

```
You ▸ kaggle.com/c/titanic → Data tab → download train.csv
```

```
You ▸ Skim the column list: Pclass, Sex, Age, Fare, Survived — which columns have missing values?
```

```
You ▸ The data dictionary explains every column — real datasets ship with one
```

```
You ▸ Fun check: average age 29.7 · survivors 342 of 891 — you'll verify both in Lab 2
```

kaggle.com/c/titanic

LIVE DEMO 3

Your co-pilot: AI-assisted debugging

You WILL hit errors tonight. Here's the routine that turns every red message into a 90-second lesson.

```
You ▸ [Paste the full error] — What does this error mean, in one sentence?
```

```
You ▸ Here's my code: [paste]. Why does it throw that error, and what's the smallest fix?
```

```
You ▸ Explain what df.groupby("Sex")["Survived"].mean() does, step by step.
```

```
You ▸ I fixed it — now explain why my fix worked, so I learn the pattern.
```

chatgpt.com

HANDS-ON LAB 1 · THE FOUR VERBS

Read the car's diary with pandas

The detection log from the case study — as a real DataFrame, in your browser. Run the **four verbs**, then answer a research question with a filter.

```python
import pandas as pd

# The car's diary: one row = one detected object
log = pd.DataFrame({
    "frame":    [1, 1, 2, 2, 2, 3, 3, 4, 4, 5],
    "object":   ["pedestrian", "car", "car", "sign", "pedestrian",
                 "car", "pedestrian", "car", "sign", "pedestrian"],
    "conf":     [0.94, 0.88, 0.91, 0.79, 0.62, 0.95, 0.58, 0.90, 0.83, 0.97],
    "dist_m":   [8.2, 15.0, 14.1, 22.4, 9.5, 13.0, 7.8, 16.2, 21.0, 6.9],
})

print(log.head())                       # 2. INSPECT: first rows
```

▶ Run | 🎧 Explain each line | ✨ Explain | 🐞 Debug | ⚡ Improve | 🤖 Mira

Python loads on first Run

```
▶ Run the code to see the output here.
```

HANDS-ON LAB 2 · CLASS 1'S LAB, GROWN UP

Titanic survival — with real pandas moves

In Class 1 you counted 12 toy rows by hand-written loops. Today: the same question with **real tools** — missing values, medians, and one groupby line that answers everything.

```python
import pandas as pd
import numpy as np

# A slice of the real Titanic data (age holes and all)
df = pd.DataFrame({
    "name": ["Braund","Cumings","Heikkinen","Futrelle","Beesley","Moran",
             "McCarthy","Palsson","Vander Planke","Nasser","Sandstrom","Bonnell"],
    "sex":  ["male","female","female","female","male","male",
             "male","male","female","female","female","female"],
    "pclass":   [3, 1, 3, 1, 2, 3, 1, 3, 3, 2, 3, 1],
    "age":      [22, 38, 26, 35, 34, np.nan, 54, 2, 18, np.nan, 4, 58],
    "survived": [0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1]
```

▶ Run | 🎧 Explain each line | ✨ Explain | 🐞 Debug | ⚡ Improve | 🤖 Mira | Python loads on first Run

```
▶ Run the code to see the output here.
```

YOUR PROJECT

Your first data task

Take a messy CSV and get your **first real numbers and chart** out of it — in Colab, where your projects live from now on.

- Load the CSV (`read_csv`) — Titanic, or a dataset from your own project
- Diagnose holes (`isnull().sum()`), then drop or fill — **and say which**
- Compute basic statistics (`describe()`, a groupby of your choice)
- Make **one chart** — with a title and labeled axes

Deliverable

A Colab notebook showing clean data, summary stats, and one plot.

Success looks like

Code runs top-to-bottom with no errors — and the chart has a title and labeled axes.

BUILD IT NOW · HANDS-ON

File → clean → stats → chart

1. Open Colab → new notebook → upload **train.csv** from Kaggle's Titanic page
2. Run the four verbs: `read_csv` → `head` → `info` → `describe`
3. Handle the Age holes (median fill) — print a line saying what you did
4. One groupby that interests you (survival by sex? class? age group?)
5. One labeled chart: `df["Age"].plot(kind="hist")` + title + axis labels
6. Runtime → **Run all** — zero errors? Submit the notebook link in Classroom.

Google Colab | pandas | ChatGPT / Claude (debug co-pilot)

Stuck on an error? That's the plan working: paste it into your assistant, understand it, fix it yourself, ask why the fix worked. Every error survived tonight is a pattern you own forever.

BENCHMARK · JUDGE AGAINST THIS

What a finished first data task looks like

🧹 Clean, disclosed

```
df["Age"] =
df["Age"].fillna(df["Age"].median())
print("Filled 177 missing ages with
median 28.0")
```

The fill is **announced**, not hidden — a reader knows exactly what was done.

📊 A stat that says something

```
df.groupby("Sex")["Survived"].mean()
# female 0.74 · male 0.19
```

Not just describe() dumped — one grouped comparison with a takeaway sentence under it.

📈 A chart a stranger can read

```
ax = df["Age"].plot(kind="hist",
bins=20,
  title="Titanic passenger ages")
ax.set_xlabel("Age (years)")
```

Title ✓ · axis labels ✓ · a sentence: "most passengers were 20–40."

The bar in one line: someone who wasn't in class tonight could open your notebook, press Run all, and understand what you found. That's a research artifact, not homework.

REFERENCE · KEY TERMS

Today's vocabulary

DataFrame — pandas' table: rows = examples, columns = variables.

Series — one labeled column; what you get from `df["col"]`.

CSV — comma-separated values: the plain-text table format every tool speaks.

Boolean indexing — filtering rows with a True/False condition: `df[df["x"] > 0.9]`.

NaN — "not a number": pandas' marker for a missing value. Find with isnull(), then drop or fill.

EDA — exploratory data analysis: the inspect-summarize-plot ritual you do before any modeling.

RECOMMENDED HOMEWORK

Before next class

1. **Finish your first data task** (clean + stats + labeled chart, Run-all clean) and submit the Colab link in Google Classroom.
2. **Full Titanic, three questions:** survival rate by sex on all 891 rows · `value_counts()` of the Embarked column · fill Age's holes with the **median** and re-run — do the stats move?
3. **Log one debugging win:** paste one error you hit + the one-sentence explanation your AI co-pilot gave + your fix.

⭐ Stretch (optional)

Enter Kaggle's **"Titanic — Machine Learning from Disaster"** for real: run a starter notebook top to bottom and submit a prediction. Data science's "Hello World" — you now know every pandas line in it.

kaggle.com · Titanic

Prefer working local? Anaconda + Jupyter is the classic setup — but Colab needs zero installs, so it stays our default. Questions → 💬 on any slide.

TODAY'S LINE TO REMEMBER

"Without data, you're just another person with an opinion."

— W. Edwards Deming

Tonight you stop being that person: four verbs, one chart, real numbers.

🔊 Speak this

WRAP-UP

Three things to keep

- Data is **rows (examples) × columns (variables)** — see the table, and code gets simple
- **Load → inspect → select → summarize** — four verbs are 80% of data work
- Real data has holes: find them, drop or fill them, and **disclose what you did**

Exit ticket: what's one column you'd add to the car's detection log — and what question would it let you ask? Post in 💬.

Next · Class 6 — the research toolbox: the tools that find the right papers and datasets for you.

BACKUP MATERIAL

Extra slides — if there's time

Deeper dives and extra practice. Skip in a tight hour; pull up on demand or for a longer session.

The four jobs of data analysis

understand · clean · engineer · validate

Data literacy

a life skill, not a job skill

Bonus quiz

one more check

BACKUP · ZOOM OUT

Data analysis has four jobs

1 · Understand

Shape, distributions, quality — today's inspect verbs. *You are here.*

2 · Clean

Missing values, outliers, duplicates, formats — today's dropna/fillna, expanded in **Class 8**.

3 · Engineer features

Build better columns from raw ones (age → age-group; time → rush-hour flag) — the craft of **Classes 7–8**.

4 · Validate

Test whether the data supports your hypothesis — statistics + charts, **Classes 9 & 14**.

All four live inside the **Data** box of your pipeline diagram — which is why that box earned your ⚠ in Class 3, and why it gets three full classes.

BACKUP · EXTENDED THINKING

Data literacy: not just for scientists

🧐 Critical reading

Where did this number come from? What's the sample? What's NOT shown? — the same skepticism you learned for LLM claims, aimed at statistics.

🎯 Evidence-based choices

From picking a phone plan to picking a college: deciding on data beats deciding on vibes — Deming's point, applied to life.

🗣️ Data storytelling

Turning numbers into a sentence others understand — the skill Class 9 trains, and the one careers are built on.

> "Learning data analysis is not just mastering a tool — it's cultivating a rational, structured way of seeing the world." The four verbs are for everyone.

BACKUP · BONUS CHECK

Quick check · tap an answer

df.groupby("Sex")["Survived"].mean() returns female 0.74, male 0.19. Which conclusion does this data support?

- Being female caused survival
- In this dataset, female passengers survived at a far higher rate — why, the table alone can't say
- 74% of passengers were female
- The 0.19 must be a data error